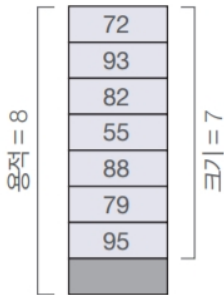
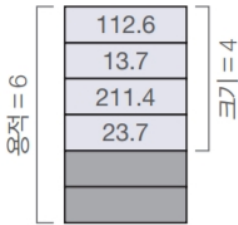

Chap 08 배열기본

Array Attributes Part 1

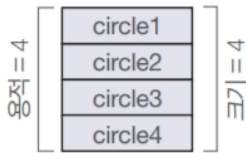
- 배열은 자료형(type), 용적(capacity), 크기(size)라는 3가지 속성이 있음



int 자료형의 배열



double 자료형의 배열



Circle 자료형의 배열

그림 8-1 배열의 속성

int A[8];

double B[6];

Circle C[4];

Array Attributes Part 2

자료형

- 배열의 자료형이란 요소(element)의 자료형을 의미
- 예를 들어 int 배열, double 배열, char 배열, Circle 배열 등이 있음
- 배열 내부의 모든 요소는 같은 자료형을 가져야 함

용적

- 배열의 용적은 보유할 수 있는 요소의 최대 수
- 이 속성은 배열이 선언된 이후에는 변경할 수 없는 상수(또는 리터럴) 값이라는 점을 유의
- 일반적으로 용적을 나타내는 상수는 대문자를 사용

Array Attributes Part 3

크기

- 배열의 크기는 어떤 시점에 존재하는 요소의 수
- 따라서 유효한 데이터의 수라고 말할수 있음
- 크기는 시간에 따라서 변함
- 10이라는 용적을 가진 배열이 있을 때, 어떤 때는 요소가 3개만 있을 수도 있고, 다른 때는 요소가 8개가 있을 수도 있음

배열 생성(1)

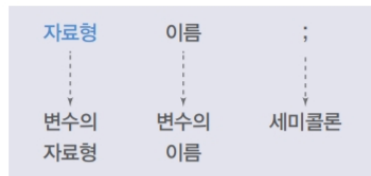
- 배열은 사용하기 전에 미리 선언해야 하며 이렇게 하면 컴파일러가 메모리의 특정 위치에 배열을 할당
- 처음 배열을 선언할 때 배열의 요소를 초기화할 수도 있음

배열 선언

- 배열 선언(array declaration)은 배열에 이름, 요소의 자료형, 배열의 용적을 지정하여 메모리에 영역을 할당하는 작업



(a) 배열 선언



(b) 변수 선언

그림 8-2 배열 선언과 변수 선언 비교

배열 생성(2)

1. 배열의 선언은 자료형, 이름, 용적, 세미콜론으로 이루어짐
2. 배열의 선언은 변수 선언과 비교하여 대괄호와 용적이 추가된 것을 확인할 수 있음
3. 배열의 용적은 상수(또는 리터럴)

배열 생성(3)

- 다음은 용적으로 리터럴 값을 사용

```
int scores [8];          double nums [6];          Circle crls  
[4];
```

```
int A[5] = {1, 2, 3, 4, 5};  
int B[5] = {10, 20, 30, 40, 50};  
A=B; // (X) LHS must be an var!!  
이유 : A 는 int pointer “상수” 라서  
좌변에는 항상 변수가 와야한다
```

```
const int CAPACITY = 8;  
int scores [CAPACITY];  
[CAPACITY];
```

```
const int CAPACITY = 4;  
Circle crls
```

배열 생성(4)

메모리 할당

- scores, nums, crls라는 배열이 선언되고 메모리에 할당된 예시

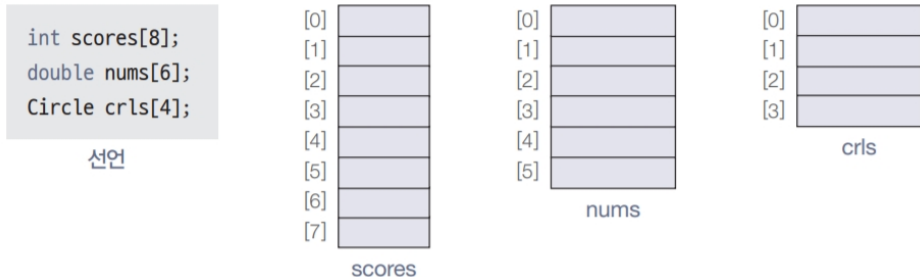


그림 8-3 서로 다른 자료형의 배열 선언 예

배열 생성(5)

초기화

- 배열 각각의 요소는 하나의 단독적인 변수와 같음
- 배열을 선언하면, 컴파일러는 배열의 자료형에 맞게 메모리 위치를 할당하며 이 때 초기화는 일반적인 변수처럼 구현

배열이 함수 내부(main 함수 등)에서 선언된 경우는 쓰레기 값으로 초기화되며 초기화에는 명시적, 암묵적, 나머지 요소의 기본 초기화가 있음

배열 생성(6)

명시적 초기화

- 배열의 초기값을 명시적으로 지정할 수 있으며 배열의 초기값은 중괄호 내부에 값들을 쉼표로 구분하여 지정

```
const int CAPACITY = 8;  
int scores [CAPACITY] = {87, 92, 100, 65, 70, 10, 96, 77};
```

배열 생성(7)

암묵적 초기화

- 다음과 같이 용적을 지정하지 않고 배열을 선언할 수 있으며 이렇게 하면 컴파일러가 알아서 용적을 설정

```
int scores [ ] = {87, 92, 100, 65, 70, 10, 96, 77};
```

배열 생성(8)

나머지 요소의 기본 초기화

- 초기화하는 값이 배열의 용적보다 크면 컴파일 오류가 발생
- 하지만 초기화하는 값이 배열의 용적보다 작은 것은 상관없음
- 이러한 경우, 배열이 선언되는 위치와 상관 없이 배열의 나머지 값은 기본값으로 채워짐

```
const int CAPACITY = 10;  
int scores [CAPACITY] = {87, 92, 100};
```

- 다음은 100개의 요소를 갖는 배열의 모든 요소를 0.0으로 명시적으로 초기화하는 예시

```
const int CAPACITY = 100;  
double anArray [CAPACITY] = {0.0};
```

배열 요소에 접근(3)

- 다음은 배열 3개의 요소에 접근하는 예시를 나타냄

scores [4] // Accessing element at location 4
nums [5] // Accessing element at location 5
crs [3] // Accessing element at location 3

선언

```
int numbers[5] = {0, 5, 10, 15, 20};  
int x;
```

[0]	0
[1]	5
[2]	10
[3]	15
[4]	20

(a) 초기화 상태

값 변경하기
numbers[2] = 67;

[0]	0
[1]	5
[2]	67
[3]	15
[4]	20

(b) 변경 후의 상태

값 추출하기
x = numbers[3];

[0]	0
[1]	5
[2]	67
[3]	15
[4]	20

(c) 추출한 요소의 값 출력하기

결과
x 15

그림 8-5 요소의 값 추출하고 변경하기

배열 요소에 접근(4)

out of range , index out of bound 오류

- 배열의 용적보다 큰 인덱스의 요소에 접근하면 out of range 오류가 발생
- 이러한 문제는 컴파일 중에 컴파일러가 찾아줄 수 없고, 실행 중에도 정확히 확인할 수 없음
- 첨자 표현식에서 인덱스는 반드시 0과 (용적 - 1) 사이의 값이어야 하며 만약 이 범위를 벗어난 인덱스를 사용하면 쓰레기 값을 추출하거나 오류가 발생
- 만약 범위를 넘는 배열의 요소를 추출해서 값을 지정하면, 데이터와 프로그램에 심각한 문제가 발생할 수 있음

runtime 에 알수있어서 미리 컴파일 전에 확인해야함

실행: 숫자 배열을 하나씩 입력 받아 거꾸로 출력하기

숫자배열요소를 하나씩!!!!

0 번째 수: 5

1 번째 수: -764

2 번째 수: 7

3 번째 수: 8

4 번째 수: 9

5 번째 수: 22

6 번째 수: -7

7 번째 수: 9

8 번째 수: 6

9 번째 수: -3

-3 6 9 -7 22 9 8 7 -764 5

숫자 배열을 거꾸로 출력하기

- 짝고 넘어가야 하는 2가지 중요한 점

1. 유효성 검사 반복문에서 변수 `size`의 범위를 1~10으로 한정했으며 반복문을 돌면서 인덱스 0~9의 요소를 출력하므로, `out of range` 오류가 발생하지 않음

키보드로부터 숫자를 읽어 들이는 반복문은 공백으로 구분된 숫자가 적힌 파일을 처리하는 것처럼 작동, `size` 변수에 의해서 정의된 횟수만큼 입력을 받으므로, 사용자가 리턴을 입력해도 계속해서 입력을 기다림(단순한 공백으로 취급). 반대로 사용자가 `size` 변수에 정의한 것보다 많은 숫자를 입력하면, 나머지는 무시, 두 번째 실행해서 마지막에 입력한 정수 9는 무시됐다는 점을 확인할 수 있음

배열의 함수 사용(1)

함수에 배열 전달

- 호출된 함수 쪽에서 호출한 함수에 있는 배열에 접근하려면, 배열의 시작 주소와 크기를 알아야 함
- 시작 주소는 `int array[ ]`(자료형, 배열의 이름, 빈 괄호)를 매개변수로 사용해서 전달받을 수 있음
- 배열의 크기는 별도의 매개변수로 전달받으면 됨

배열의 함수 사용(2)

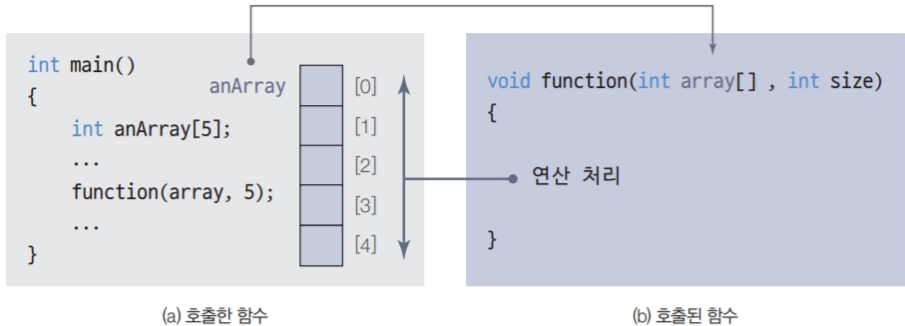


그림 8-11 함수에 배열 전달하기

왜 size정보를 전달해야 할까?

array name?

- pointer constant, 포인터상수

모든 포인터는 4바이트
그리고 이거 피피티 이 페이지만
이상하니까 나중에 수정 ㄱ ㄱ

int i ; // i is int var

int* ip; // ip is int* var, 포인터변수

int j;

ip = &j; // ip is a var, and the value can be
changed

int A[5]; // A is int pointer constant. 값 &A[0]

i = A[5]; // A is int pointer constant. 값 &A[0]

학기초 테스트 프로그램

- main() 에서
int 배열 크기 5짜리 선언하면서 요소를 적당한 값으로
초기화 할 것
함수 funcmin()에 배열을 전달하기
funcmin() 에서 받은 최소값 출력하기
- funcmin()은 배열을 받아서, 배열 요소의 최소값을 찾아서 리턴하는
함수
배열의 요소에 접근할 때 여러가지 방법이 있다.
가능한 int pointer변수를 사용할 것.

텍스트

Soln.

```
int funcmin(int* ip, int  
size)
```

```
int main() {  
    int A[5] = {0, -9, 33,  
-45, -19};  
    int res = funcmin(A, 5);  
    cout << "최소값은 : "<<  
res << endl;  
}
```

```
int funcmin(int B[], int  
size);
```

```
int main() {  
    int A[5] = {0, -9, 33,  
-45, -19};  
    int res = funcmin(A,  
5);  
    cout << "최소값은 : "  
<< res << endl;  
}
```

배열의 함수 사용(8)

- C++은 함수 자료형을 리턴할 수 없으며 다음과 같은 함수 프로토타입은 만들 수 없다는 의미

```
type [ ] function(const type array [ ], int size); Not allowed
```

- 배열을 리턴하는 방법은 **dynamic memory allocation**을 활용한다
동적 메모리 할당 !

```
// array will not change.  
void function(const type array [ ], int size);  
  
// array will change.  
void function(type array [ ], int size);
```